MODULE AWD · VALIDATION 1 · SÉANCE 4

Clinique+

Gestion d’une clinique en microservices : du rendez-vous à la facture, sans ressaisie.

Firas Ferchichi · Doua Jlassi · Emna Mejri · Wassim Becharguia · Sahar Ouslati · Dhia Mouha

ESPRIT · Année universitaire 2026-2027

L’ÉQUIPE

Un microservice par membre, un service commun

Membre	Microservice	Base	Partie de groupe présentée
Firas Ferchichi	patient-service	MySQL	Problématique + service commun
Doua Jlassi	personnel-service	MySQL	Acteurs et fonctionnalités
Emna Mejri	rendezvous-service	PostgreSQL	Architecture globale
Wassim Becharguia	consultation-service	MongoDB	Sécurité, découpage, données
Sahar Ouslati	pharmacie-service	PostgreSQL	Communications, technologies, Git
Dhia Mouha	facturation-service	PostgreSQL	Planning et risques
Toute l’équipe	notification-service	MongoDB	Présenté par Firas

PROBLÉMATIQUE

Accueil, médecins, pharmacie et caisse ne se parlent pas

Agenda

Doubles réservations, rendez-vous oubliés sans rappel.

Dossier

Historique dispersé ; le médecin ne voit pas les allergies.

Pharmacie

Ordonnances recopiées, ruptures de stock découvertes trop tard.

Caisse

Factures et part mutuelle calculées à la main.

NOTRE RÉPONSE

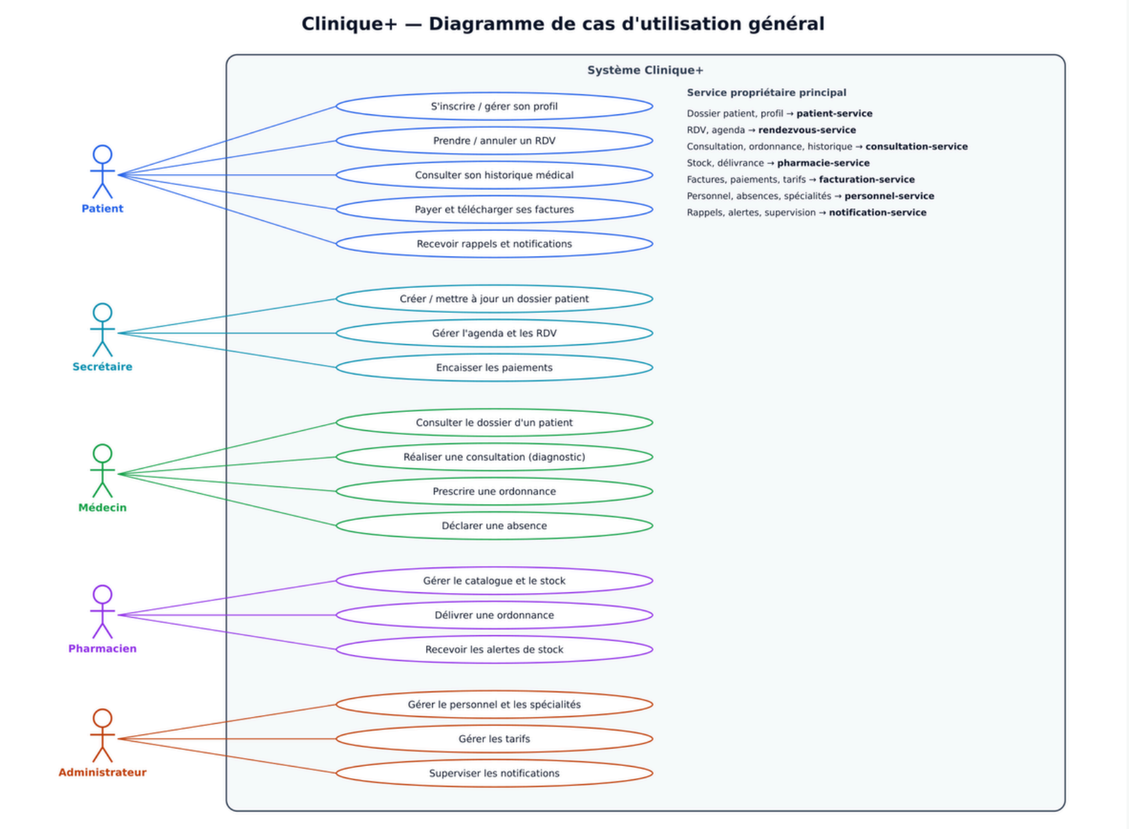
Un parcours patient unique — inscription → RDV → consultation → ordonnance → délivrance → facture — où chaque étape déclenche automatiquement la suivante.

ACTEURS ET FONCTIONNALITÉS

5 rôles Keycloak, 11 user stories

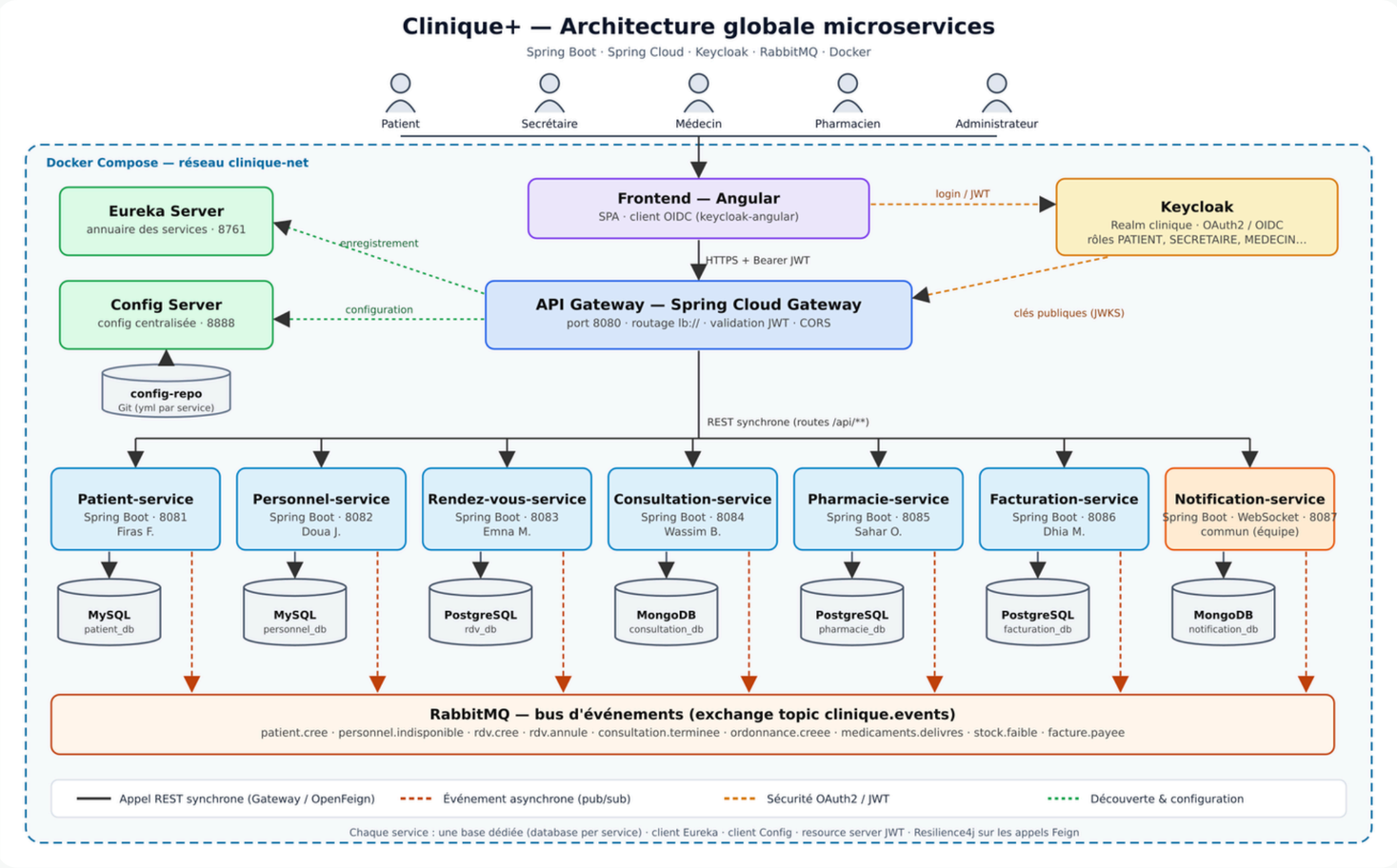
Acteur	Il peut…
Patient	réserver, voir son historique, payer
Secrétaire	gérer dossiers, agenda, encaissement
Médecin	consulter, diagnostiquer, prescrire
Pharmacien	gérer le stock, délivrer
Administrateur	personnel, tarifs, supervision

Hors périmètre : téléconsultation, paiement en ligne réel, hospitalisation.



ARCHITECTURE GLOBALE

Un point d’entrée, 7 services, 7 bases



API Gateway

Routage par chemin vers lb://service, JWT vérifié.

Eureka + Config

Découverte des instances ; un fichier de config par service et par profil.

Keycloak

Realm « clinique », 5 rôles, OAuth2 / OIDC.

RabbitMQ

Exchange topic, 9 événements métier.

Docker Compose

20 conteneurs sur le réseau clinique-net.

SÉCURITÉ

Le jeton est vérifié deux fois : Gateway et service

1

Login Keycloak

Authorization Code + PKCE depuis Angular → JWT avec les rôles.

2

Gateway

Bearer JWT obligatoire sur /api/** ; sinon 401.

3

Microservice

Resource server : revalide via JWKS, rôles → @PreAuthorize.

4

Appels Feign

Le jeton de l'utilisateur est propagé au service appelé.

Exemple : seul un **MEDECIN** peut appeler POST /api/consultations/{id}/ordonnance, sinon 403.

DÉCOUPAGE ET DONNÉES

7 contextes bornés, chacun sa base

Service	Capacité métier	Base	Pourquoi cette base
patient	Identité du patient	MySQL	Structuré, stable, CIN unique
personnel	RH médicales	MySQL	Référentiel, jointures horaires
rendezvous	Planification	PostgreSQL	Index unique partiel : pas de double RDV
consultation	Acte médical	MongoDB	Document imbriqué, forme variable
pharmacie	Stock	PostgreSQL	ACID + verrou : jamais de stock négatif
facturation	Finances	PostgreSQL	Montants exacts, intégrité
notification	Communication	MongoDB	Journal volumineux, purge TTL

Aucune base partagée : entre domaines on ne garde que des identifiants (patientId, medecinId) et on passe par l'API ou les événements.

COMMUNICATIONS

Synchrone si on attend, sinon un événement

8 appels synchrones

OpenFeign + Eureka + circuit breaker Resilience4j, chaque appel a un fallback.

```
rendezvous → patient
GET /api/patients/{id}
```

9 événements asynchrones

RabbitMQ, exchange topic clinique.events, une file durable par consommateur.

```
consultation.terminee
→ facturation, rendezvous
```

EXEMPLE : LE MÉDECIN CLÔTURE UNE CONSULTATION

PATCH /terminer → 200 immédiat → publie consultation.terminee → Facturation crée la facture (lit la couverture en Feign) → Rendez-vous passe à TERMINE

TECHNOLOGIES ET ORGANISATION

Stack imposée, dépôt structuré, tâches suivies

Couche	Choix
Back	Java 21, Spring Boot 4.1
Microservices	Spring Cloud 2025.1 : Gateway, Eureka, Config, OpenFeign
Résilience	Resilience4j
Messagerie	RabbitMQ 4, Spring AMQP
Sécurité	Keycloak 26, OAuth2 Resource Server
Bases	MySQL 8.4, PostgreSQL 17, MongoDB 8
Front	Angular 22
Déploiement	Docker Compose

Dépôt Git

Backend/ (un projet par service), Frontend/, Documentation/, README, docker-compose.

Branches

main ← develop ← feature/<service> ; PR relue par un autre membre.

Suivi

GitHub Projects : Kanban, une issue par tâche.

PLANNING ET RISQUES

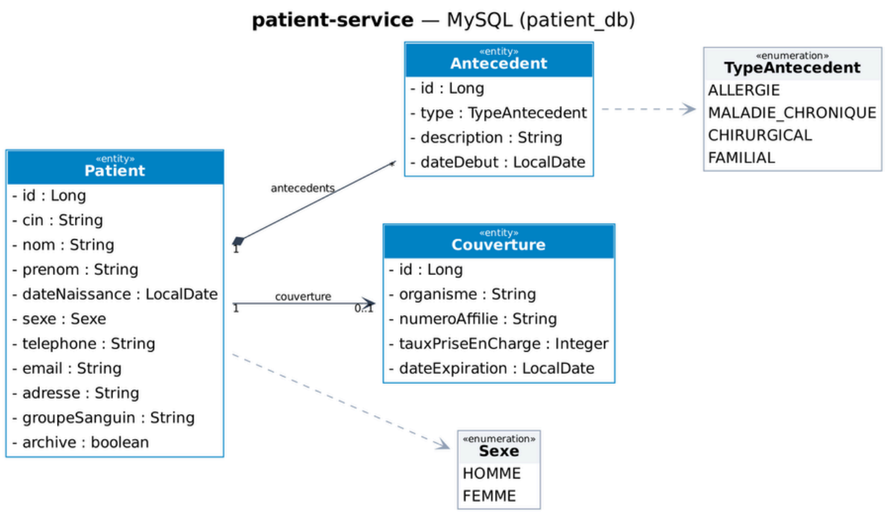
Du CRUD aux événements, puis à l’intégration

Séance 4	Sem. 5–6	Sem. 7–8	Sem. 9	Sem. 10–11	Sem. 12
Conception validée	Entités + CRUD REST	Feign + événements	Keycloak + Notification	Écrans Angular + Docker	Tests et démo finale

Risque	Mesure prévue
Contrat d’API qui change	Contrats figés dans le dossier, Swagger par service
Panne en cascade	Circuit breaker + fallback sur chaque appel Feign
Message traité deux fois	Consommateurs idempotents (consultationId unique)
Keycloak complexe	Realm exporté dans le dépôt, sécurité désactivable en local
Membre absent	Revue croisée : chaque service connu de deux membres

MICROSERVICE 1 · PORT 8081 · MYSQL PATIENT_DB

patient-service — Firas Ferchichi



Endpoint	Codes
POST /api/patients	201 400 409
GET /api/patients/{id}	200 404
PUT /api/patients/{id}	200 404
DELETE /api/patients/{id} (archive)	204 404
GET /api/patients/{id}/couverture	200 404

Appelé par (Feign) : rendezvous, consultation, facturation, notification

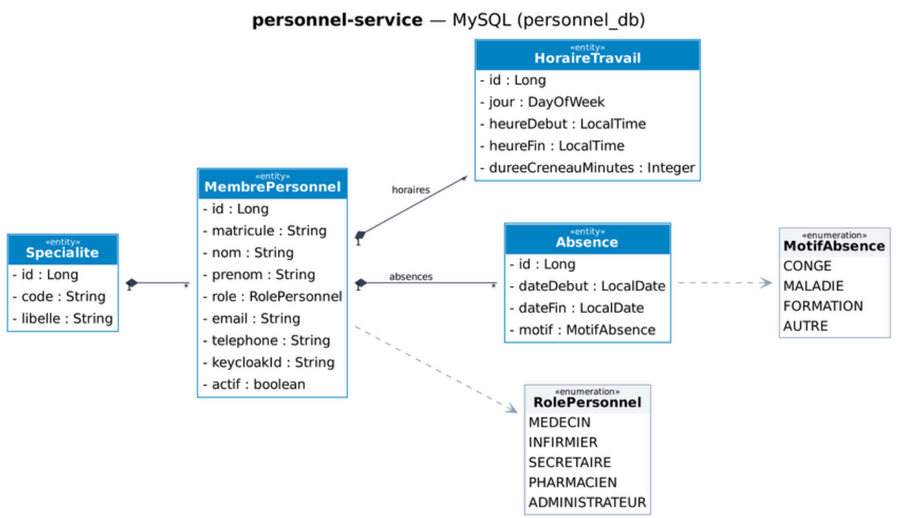
Publie : patient.cree → notification (bienvenue)

Base : MySQL, données stables, CIN unique (409 si doublon)

Panne : ses appelants basculent sur leur fallback

MICROSERVICE 2 · PORT 8082 · MYSQL PERSONNEL_DB

personnel-service — Doua Jlassi



Endpoint	Codes
POST /api/personnel	201 400 409
GET /api/personnel/medecins?specialite=	200
PUT /api/personnel/{id}/horaires	200 404
GET .../medecins/{id}/disponibilites	200 404
POST /api/personnel/{id}/absences	201 404

Appelé par (Feign) : rendezvous, pour les plages de travail

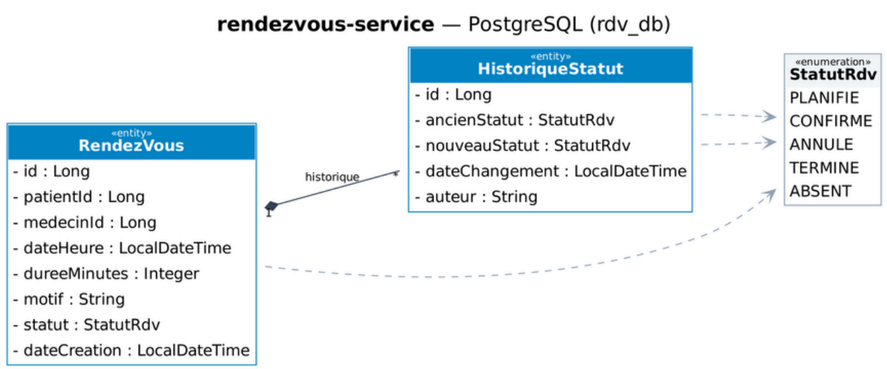
Publie : personnel.indisponible → rendezvous, notification

Base : MySQL, référentiel relationnel

Panne : Rendez-vous répond 503 « réessayer » ; les RDV existants restent lisibles

MICROSERVICE 3 · PORT 8083 · POSTGRESQL RDV_DB

rendezvous-service — Emna Mejri



Endpoint	Codes
POST /api/rendez-vous	201 404 409 503
GET /api/rendez-vous/creneaux-libres	200 503
GET /api/rendez-vous/{id}	200 404
PATCH /api/rendez-vous/{id}/confirmer	200 409
PATCH /api/rendez-vous/{id}/annuler	200 409

Appelle (Feign) : patient (existe ?), personnel (plages)

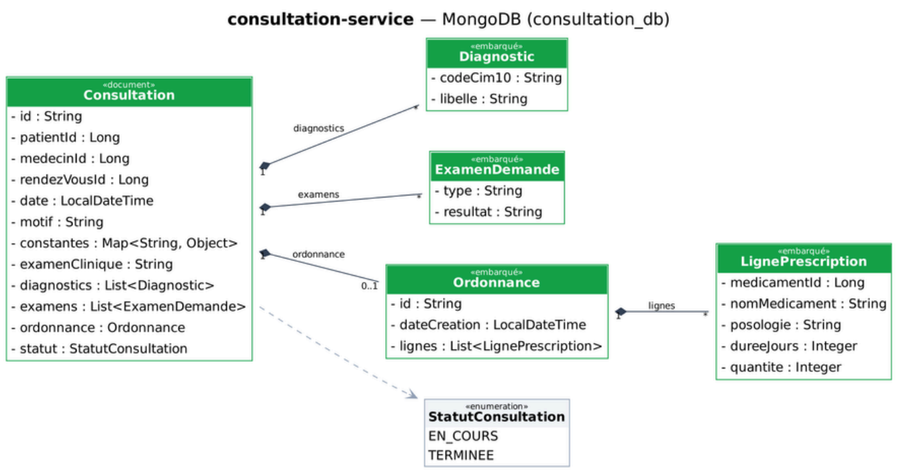
Publie : rdv.cree, rdv.annule · **consomme** : personnel.indisponible, consultation.terminee

Base : index unique partiel → 409 si créneau pris

Panne de patient : circuit ouvert → 503, réservation refusée

MICROSERVICE 4 · PORT 8084 · MONGODB CONSULTATION_DB

consultation-service — Wassim Becharguia



Endpoint	Codes
POST /api/consultations	201 404 503
PUT /api/consultations/{id}	200 409
POST .../{id}/ordonnance	201 400
PATCH .../{id}/terminer	200 409
GET /api/consultations/patient/{id}	200

Appelle (Feign) : patient (allergies), rendezvous (RDV), pharmacie (catalogue)

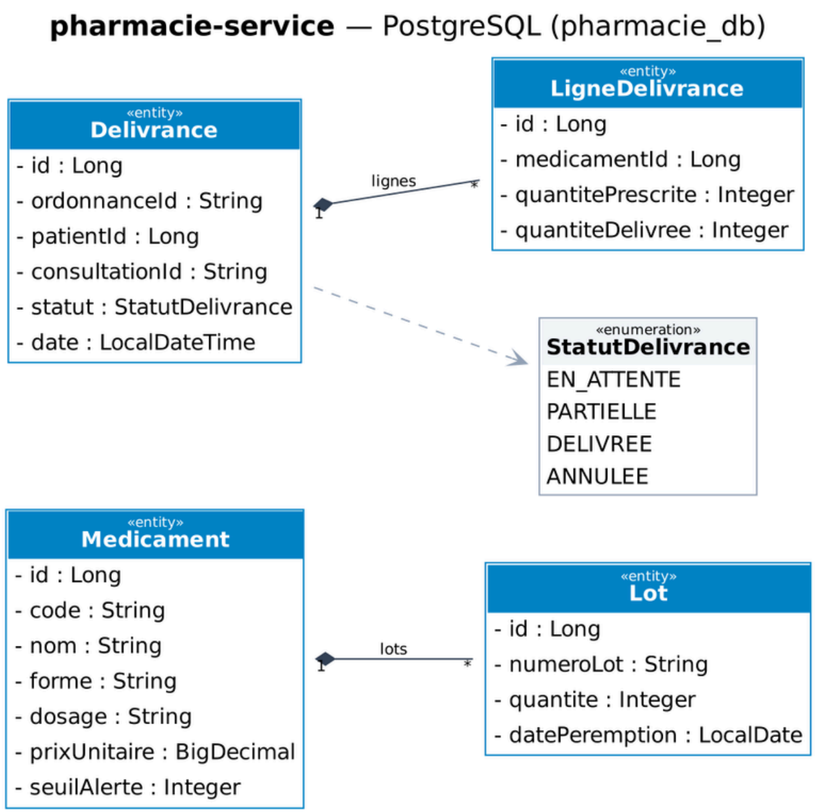
Publie : consultation.terminee, ordonnance.creee

Base : MongoDB, un document imbriqué par consultation

Panne de patient : résumé en cache + bandeau « données partielles »

MICROSERVICE 5 · PORT 8085 · POSTGRESQL PHARMACIE_DB

pharmacie-service — Sahar Ouslati



Endpoint	Codes
GET /api/medicaments?nom=	200
POST /api/medicaments/{id}/lots	201 404
GET /api/medicaments/alertes	200
GET /api/delivrances?statut=EN_ATTENTE	200
POST /api/delivrances/{id}/valider	200 409

Consomme : ordonnance.creee → crée une délivrance EN_ATTENTE

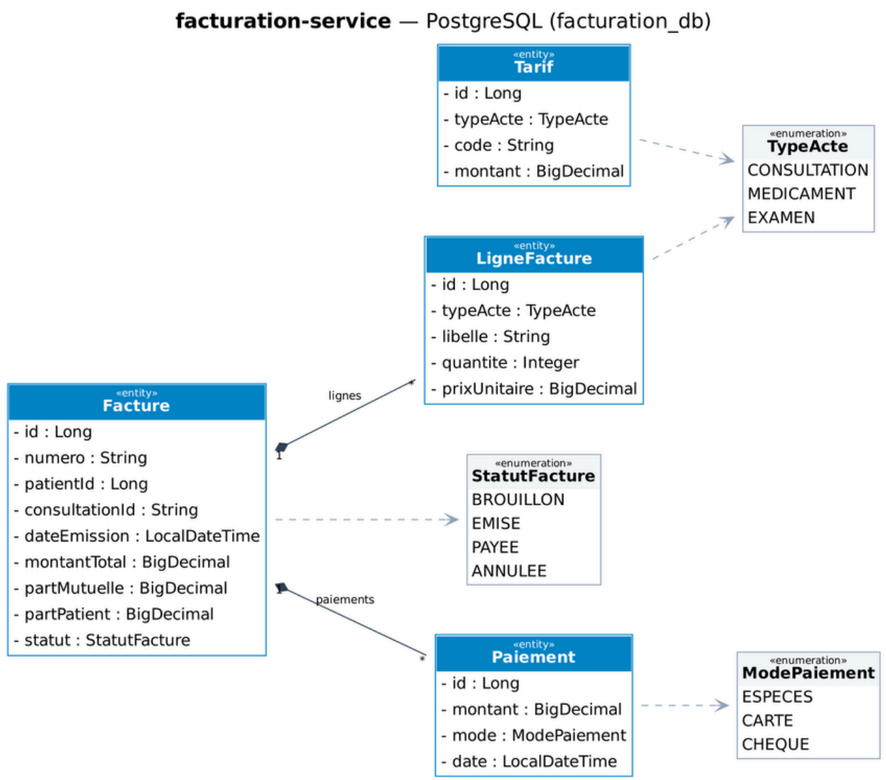
Publie : medicaments.delivres → facturation, stock.faible → notification

Appelle consultation (relire l'ordonnance) ; **appelé par** consultation (catalogue)

Base : ACID + SELECT FOR UPDATE → 409 si stock insuffisant

MICROSERVICE 6 · PORT 8086 · POSTGRESQL FACTURATION_DB

facturation-service — Dhia Mouha



Endpoint	Codes
GET /api/factures?patientId=	200
GET /api/factures/{id}	200 404
POST /api/factures/{id}/paiements	201 409
PATCH /api/factures/{id}/annuler	200 409
GET /api/factures/{id}/pdf	200 404

Consomme : consultation.terminee (crée la facture), medicaments.delivres (ajoute les lignes)

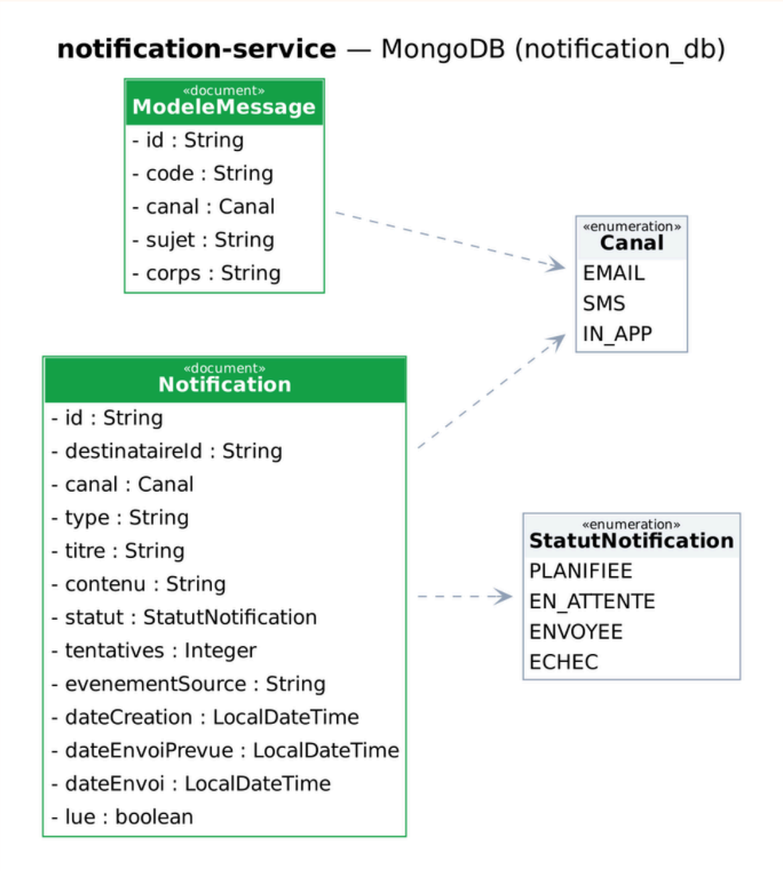
Appelle patient : couverture → part mutuelle / part patient

Publie : facture.payee → notification (reçu)

Panne de patient : taux 0 %, facture en BROUILLON, recalculée

MICROSERVICE COMMUN · PORT 8087 · MONGODB NOTIFICATION_DB

notification-service — toute l’équipe



Endpoint	Codes
GET /api/notifications/me	200 401
PATCH /api/notifications/{id}/lue	200 404
GET /api/notifications?statut=ECHEC	200 403
POST /api/notifications/{id}/renvoyer	202 404
WS /ws/notifications (STOMP)	—

Consomme 7 événements : patient, personnel, rdv, ordonnance, stock, facture

Technologie avancée : RabbitMQ, WebSocket temps réel, rappels J-1 planifiés

Appelle patient (contact) ; personne ne l’appelle en synchrone

Panne : messages gardés en file ; échec d’envoi → EN_ATTENTE + retry

Merci — vos questions

7

microservices, 7 bases

8

appels synchrones avec fallback

9

événements RabbitMQ

10

projets Spring Boot qui démarrent

Démo du dépôt : structure, projets créés, historique des commits.